

Project Report: Multi-Exchange Crypto Arbitrage Detector

Yıldız Technical University – MTM4692 Applied SQL

Students: Gülcan Özer & Ecem Demir

Student IDs: 22025078 & 21025006

Date: June 2026

GitHub Repository Link: https://github.com/ozergulcan/Crypto_Arbitrage_Detector.git

Video: https://drive.google.com/file/d/1jRopAYniEwAh1hGxn5wZD9NVbPhFKxX4/view?usp=drive_link

1. Introduction

The project aims to detect arbitrage opportunities for Bitcoin across four cryptocurrency exchanges: Binance, Coinbase, Bitfinex, and Bitstamp. Using historical daily price data and trading fees, the system calculates net profit percentages and presents the results through a web dashboard.

The initial proposal described a MySQL-based solution with a focus on SQL queries and a simple frontend. During implementation, several adjustments were made to improve portability, performance, and user experience.

2. Final Database Design

The final schema (SQLite) consists of:

- exchanges – exchange names, locations, reliability scores.
- assets – only BTC (asset_id = 1).
- market_data – daily OHLCV data per exchange (close_price is the key field for arbitrage).
- trading_fees – maker/taker fees per exchange.
- arbitrage_logs – stores only profitable opportunities (net profit > 0) after fees.
- staging tables – raw CSV data from each exchange before cleaning.

Relationships: one-to-many from exchanges to market_data, one-to-one from exchanges to trading_fees.

3. Implementation Phases

- **Schema creation** (01_create_schema.sql) – defines all tables.
- **CSV loading** (02_load_data.py) – imports raw data into staging tables.
- **Data cleaning and insertion** (03_clean_and_load.sql) – transforms and moves data into market_data.
- **Arbitrage view** (04_arbitrage_analysis.sql) – creates arbitrage_view for gross profit calculations.
- **Logging profitable opportunities** (05_save_logs.sql) – calculates net profit using taker fees and inserts into arbitrage_logs.
- **Web application** (app.py + index.html) – Flask backend with Bootstrap dashboard, KPI cards, and Chart.js visualizations.

5. What Changed from the Initial Plan

- **Database engine:** MySQL → SQLite.
Reason: SQLite requires no separate server, making the project fully portable and easier to distribute.
- **Arbitrage calculation:** Originally relied on a separate `arbitrage_view`.
Change: Embedded the logic as CTEs inside `app.py`.
Reason: Avoids dependency on a persistent view and simplifies deployment (just run the Python script).
- **Web interface:** From a minimal Plotly-based chart page to a Bootstrap dashboard.
New features: KPI cards (total opportunities, max/avg net profit), top 5 most frequent profitable routes, exchange fee table, and a horizontal bar chart (Chart.js) for route distribution.
- **Data visualization library:** Replaced Plotly with Chart.js.
Reason: Chart.js is lighter, easier to integrate with Bootstrap, and rendered faster.
- **Output focus:** Initially only listed arbitrage logs.
Added: Summary statistics and aggregated insights to help users quickly grasp the most valuable patterns.

6. Conclusion

The final system successfully identifies historical arbitrage opportunities between four Bitcoin exchanges, taking into account taker fees. The web dashboard provides a clear overview of:

- Total profitable opportunities found.
- Maximum and average net profit percentages.
- Most frequent profitable exchange pairs (e.g., Coinbase → Bitstamp).
- Exchange fee comparison.
- Detailed list of the top 100 opportunities.

The project demonstrates how SQL can be used to integrate heterogeneous financial data, perform analytical calculations, and feed results into a user-friendly interface. Future work could include real-time data ingestion, support for multiple cryptocurrencies, and automated trade simulation.